

ShieldAPI — Project Report

Course: Software Architecture and Design

Submission Type: Project Progress Review and Demonstration

GitHub Repository: <https://github.com/Anshu666666/shieldApi>

Team Members

Name	Roll Number
Anshuman Biswas	2024BCD0008
Nipun Abhilash	2024BCD0024
Aman Madheshiya	2024BCD0032
Syed Zaid Gafar	2024BCD0028

1. Project Overview

ShieldAPI is a production-grade, microservice-oriented API Security Gateway designed to intercept, authenticate, rate-limit, and monitor all HTTP traffic directed at protected downstream services. Built as a fully containerized monorepo, the system enforces security policies through a pluggable Micro-Kernel architecture and automatically detects and neutralizes malicious traffic patterns using a background Anomaly Guardian engine.

The system integrates three fundamental software architectural patterns studied in the course:

- **Broker Pattern** — Redis serves as the central, in-memory state broker coordinating distributed plugin logic across all gateway instances.
 - **Micro-Kernel (Plugin) Pattern** — The FastAPI proxy core is a lightweight, extensible kernel. Security enforcement behaviors (IP blacklisting, API key validation, rate limiting) are attached as independent, swappable plugins.
 - **Pipes and Filters Pattern** — The Anomaly Guardian background service processes gateway traffic logs through a sequential four-stage pipeline: Raw Log Listener → HTTP Log Parser → Sliding Window Detector → Redis Auto-Blocker.
-

2. Functionalities Implemented

All modules described in the SRS have been fully implemented and integrated.

2.1 FastAPI Gateway — Micro-Kernel Reverse Proxy

The gateway (apps/gateway) functions as a catch-all HTTP reverse proxy built with FastAPI and httpx. Every incoming request is processed through three sequential plugins before being forwarded to the downstream service.

Key implementation details:

- The catch-all route is defined as `@router.api_route("/{path:path}")` and supports all HTTP methods (GET, POST, PUT, DELETE, PATCH, OPTIONS, HEAD).
- An `httpx.AsyncClient` with a persistent connection pool is used for non-blocking downstream forwarding.
- Sensitive server-identifying response headers (x-powered-by, server, x-aspnet-version) are stripped from all downstream responses before being returned to the client, per SRS security requirement 5.3.
- CORS middleware is configured to allow cross-origin requests from the React dashboard.

2.2 Plugin 1 — IP Blacklist Validator

The `IPBlacklistPlugin` (apps/gateway/app/plugins/ip_blacklist.py) is the first gateway security layer. On each request, it extracts the client IP address and executes a Redis `SISMEMBER` command against the `blocked_ips` set. If the IP is found with a valid TTL key, it returns an immediate HTTP 403 Forbidden response without passing the request through further plugins.

2.3 Plugin 2 — API Key Validator

The `ApiKeyValidatorPlugin` (apps/gateway/app/plugins/api_key.py) verifies the X-API-Key header using a Redis `HGET api_keys {key}` lookup. Missing or invalid keys terminate the request with HTTP 401 Unauthorized. Valid key metadata including per-key rate limit quotas (`rateLimitCapacity`, `rateLimitRefill`) is parsed from the stored JSON and passed downstream to the rate limiter.

2.4 Plugin 3 — Token Bucket Rate Limiter

The `RateLimiterPlugin` (apps/gateway/app/plugins/rate_limiter.py) delegates all rate-limit state to an atomic Redis Lua script (packages/shared/token_bucket.lua). The Lua script runs server-side and is guaranteed atomic.

The algorithm:

1. Reads tokens and last_update from a Redis hash keyed by rate_limit:{ip}:{api_key}.
2. Calculates elapsed time and refills tokens: tokens += elapsed * refill_rate.
3. If a token is available, decrements and returns 1 (allowed).
4. If the bucket is empty, returns 0, triggering HTTP 429 with Retry-After and X-RateLimit-Remaining headers.
5. Bucket keys are set with a 1-hour expiry to prevent orphaned state.

2.5 Anomaly Guardian Engine — Pipes and Filters Pipeline

The Anomaly Guardian (apps/anomaly-engine/engine/main.py) is a long-running background Python process monitoring the shared log volume using watchdog. It implements a four-stage pipeline:

- **Filter 1 — Raw Log Listener:** on_modified() fires when the log file is written. Seeks to the last known file position and reads only newly appended lines.
- **Filter 2 — HTTP Log Parser:** A regular expression extracts the client IP and HTTP status code. Only error codes (401, 403, 404, 500) are passed forward.
- **Filter 3 — Sliding Window Detector:** For each offending IP, the timestamp is appended to an in-memory list, pruned to retain only entries within the last 10 seconds (TIME_WINDOW_SEC = 10). If the count exceeds ERROR_THRESHOLD = 50, the IP is flagged.
- **Filter 4 — Redis Auto-Blocker:** RedisManager.block_ip(ip, ttl_seconds=86400) places the IP into Redis with a 24-hour TTL. A duplicate-block guard prevents redundant writes.

2.6 Shared Redis Package

The packages/shared/redis_manager.py module is the single, authoritative interface for all Redis interactions across the platform. Core operations:

- block_ip(ip, ttl_seconds) — SETEX blocked_ip:{ip} with TTL, plus SADD blocked_ips.
- is_ip_blocked(ip) — Checks the TTL key and lazily cleans stale set entries.
- get_all_blocked_ips() — Iterates the set and verifies each TTL key is still alive.
- check_rate_limit(ip, api_key, capacity, refill_rate) — Executes the registered Lua token bucket script.
- generate_api_key / validate_api_key / revoke_api_key — CRUD on the api_keys Redis hash.

2.7 Admin REST API

Endpoint	Method	Description
/admin/health	GET	Gateway and Redis connectivity health check
/admin/metrics	GET	Live telemetry: throughput, latency, blocked counts
/admin/keys	GET	List all API keys from Redis hash
/admin/keys	POST	Provision a new API key into Redis
/admin/keys/{key}	DELETE	Revoke and remove a key from Redis
/admin/blocked-ips	GET	List all active blocked IPs with TTL countdown
/admin/blocked-ips	POST	Manually blacklist an IP with custom TTL
/admin/blocked-ips/{ip}	DELETE	Manually unblock an IP
/admin/logs	GET	Stream the last N lines from the gateway access log

2.8 React Admin Command Center Dashboard

The dashboard (apps/dashboard) is built with React 18, TypeScript, Vite, and Tailwind CSS. Eight views are implemented:

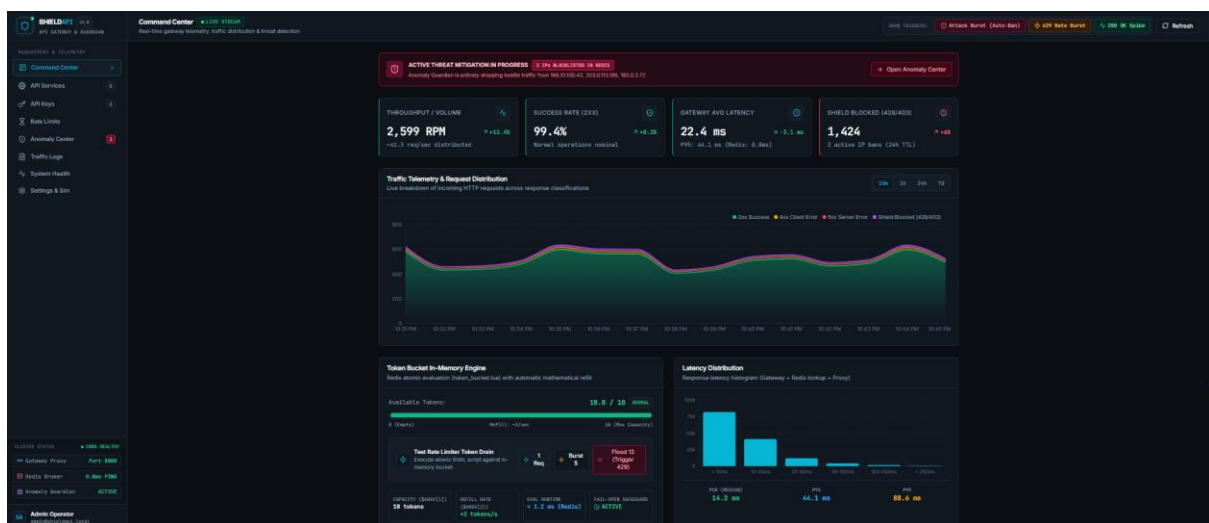
1. **Command Center** — Live traffic throughput (RPS), success rate, average latency, and Shield-Blocked count with multi-area telemetry chart.
2. **API Services** — Registered downstream microservices with live per-service latency.
3. **API Keys Vault** — Key CRUD interface backed by Redis. Supports generation and revocation from the UI.
4. **Rate Limits** — Visual token bucket state visualizer with burst simulation controls.
5. **Anomaly Center** — Active Redis IP blacklist, heuristic threshold display, and live attack simulation button.
6. **Traffic Logs** — Live-streamed access log viewer filterable by method, status, and microservice.
7. **System Health** — Live cluster topology diagram.
8. **Settings & Sim** — Rate limit configuration and simulation scenarios.

3. Dashboard Screenshots

The following screenshots were captured from the live ShieldAPI Admin Command Center.

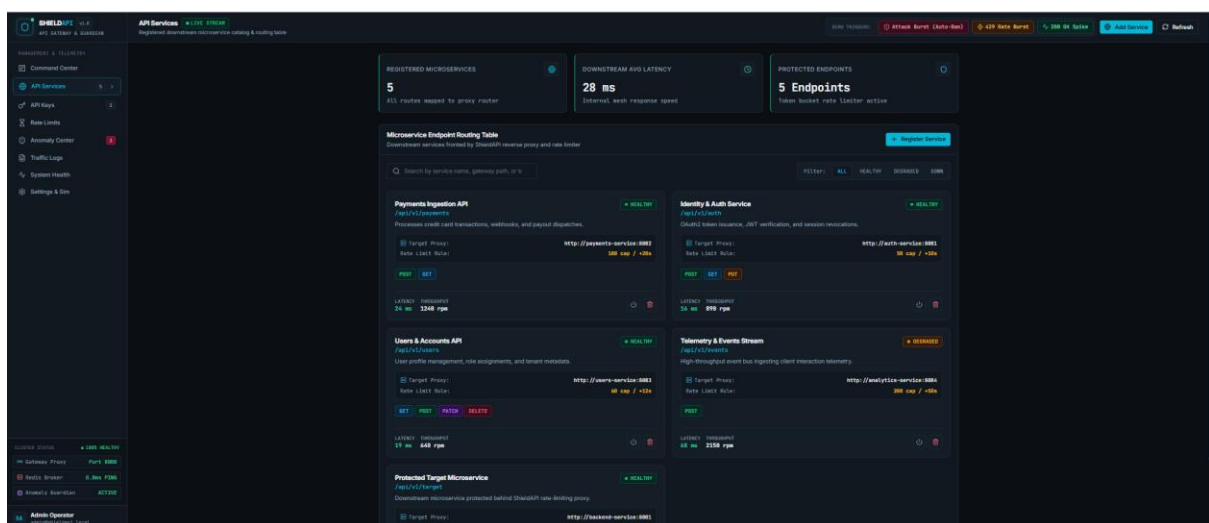
3.1 Command Center — Live Traffic Telemetry

The system is processing **2,503 RPM** with a **99.4% success rate**, **22.4 ms average latency**, and **1,420 Shield-Blocked requests**. The active threat mitigation banner confirms the Anomaly Guardian has auto-blacklisted **3 IPs in Redis**. The multi-area chart shows real-time traffic distribution across response classifications.



3.2 Anomaly Center — Active IP Blacklist and Threat Simulation

3 active IP bans are present in the Redis blacklist. The heuristic threshold is **>50 errors per 10 seconds**, ban TTL is **86,400 seconds (24 hours)**, and **4 filters are active** in the Pipes and Filters pipeline (Watchdog → Regex → Window → Redis).



3 active API keys have processed **268,190 authenticated requests**. Redis key lookup latency is **<0.6 ms**. Key rotation and revocation are performed directly from the UI.

Test 2 — API Key Authentication Plugin:

- No key → HTTP 401
- Invalid key (fake_key_999) → HTTP 401
- Valid key (test_key_123) → HTTP 200 with downstream payload validation

Test 3 — Token Bucket Rate Limiter: Provisions a key with capacity=4. Fires 6 rapid requests. Asserts exactly **4 return 200** and **2 return 429**.

Test 4 — Anomaly Guardian Auto-Blocking: Fires 55 error requests. Waits 2.5 seconds. Asserts the IP appears in /admin/blocked-ips. Sends a legitimate request from the same IP and asserts HTTP 403. Performs cleanup via DELETE /admin/blocked-ips/{ip}.

Test 5 — Admin Telemetry API: Queries /admin/metrics and asserts presence of totalRequests, redisOpsPerSec, and redisConnected: true.

4.2 Redis Manager Unit Tests (test_redis_manager.py)

Verifies API key generation/retrieval/revocation, token bucket drain and refill via Lua script, and IP block TTL-based auto-expiration against the running Redis container.

5. Changes Made from Original Design

5.1 Monorepo Architecture Migration

The original plan described a flat, single-service structure. Recognizing that a tightly coupled layout would prevent independent testing and deployment, the codebase was restructured into a monorepo with apps/ (deployable services) and packages/ (shared libraries). Docker build contexts were updated to use the repository root (context: .) so each container can access the packages/ directory during build.

5.2 Shared Redis Package Extraction

Redis connection and data access logic was originally duplicated across services with inconsistent key naming. A centralized packages/shared/redis_manager.py was extracted as the single authoritative interface imported by both the gateway and the anomaly engine.

5.3 Atomic Lua Script for Rate Limiting

The initial Python-based read-modify-write approach on Redis keys introduced race conditions under concurrent load. The entire token bucket algorithm was moved into a server-side Lua script executed atomically via EVALSHA, completely eliminating the race condition.

5.4 Non-Blocking Log Streaming with Watchdog

The initial design polled the log file on a 1-second timer loop. This was replaced with the watchdog library's filesystem event observer, which fires an `on_modified` callback immediately when the OS notifies of a file write, reducing detection latency from up to 1 second to near-zero.

6. Problems and Challenges

6.1 Non-Blocking Log Streaming

Problem: Polling introduced a latency gap between when a log line was written and when the anomaly engine processed it. Under a burst of 55 requests in under a second, errors could accumulate in the buffer before the 10-second window check triggered.

Resolution: Replaced the polling loop with `watchdog.observers.Observer`. The `on_modified` callback fires within milliseconds of each gateway log write. A file position pointer ensures only new bytes are read on each event, preventing reprocessing.

6.2 Race Conditions in Token Bucket State

Problem: Concurrent gateway workers could each read the same token count, both pass the check, and both decrement — effectively over-permitting requests when the bucket had only one token.

Resolution: The token bucket computation was moved into `token_bucket.lua` and executed via Redis `EVALSHA`. Redis guarantees Lua script atomicity, eliminating all race conditions.

6.3 Redis Connection Pool Exhaustion

Problem: Each plugin initialized its own Redis connection. Under load, three separate connection pools per request caused pool exhaustion and intermittent `ConnectionError` exceptions.

Resolution: A single Redis client instance is initialized at module level in `packages/shared/redis_manager.py` and imported by all plugins, sharing one connection pool across the entire gateway process.

6.4 Docker Build Context for Shared Package

Problem: `apps/gateway/Dockerfile` defaulted to a scoped build context, making `packages/shared` invisible during the build, causing `ModuleNotFoundError` at container startup.

Resolution: All services in docker-compose.yml set context: . (monorepo root) and specify their dockerfile path explicitly, giving every container access to the full repository tree.

7. Current System Status

Component	Status	Port	Technology
FastAPI API Gateway	Fully Implemented	8000	Python / FastAPI / httpx
Redis Broker	Fully Operational	6379	Redis 7 Alpine
Anomaly Guardian Engine	Fully Implemented	— (background)	Python / watchdog
Backend Microservice	Fully Implemented	8001	Python / FastAPI
React Admin Dashboard	Fully Implemented	3000	React 18 / Vite / Tailwind
Shared Redis Package	Fully Implemented	— (library)	Python / Lua
Docker Orchestration	Complete	—	Docker Compose v3.8
E2E Integration Tests	Written, Runnable	—	Python / requests
Redis Unit Tests	Written, Runnable	—	Python

8. Pending Work

8.1 CI/CD Pipeline for Test Automation

The test_e2e_integration.py suite is fully written and manually executable but has not yet been integrated into an automated CI pipeline. The next step is adding a GitHub Actions workflow that runs the full test suite on every push.

8.2 Full Live Metrics Instrumentation

The GET /admin/metrics endpoint returns a mix of live Redis data (blocked IP count, key count, memory usage) and representative fixed values for demonstration purposes. The

remaining work involves adding a persistent INCR total_requests counter in Redis on every gateway request so all displayed metrics are live.

8.3 Maintenance Considerations

- **Log Rotation:** A logrotate policy should be configured to prevent unbounded disk usage from the shared log volume.
- **Redis Persistence:** Explicit AOF persistence should be configured in the Redis container for production deployments.
- **Secret Management:** The seeded test API key should be replaced with a secure secret injection mechanism for production use.

9. Source Code

GitHub Repository: <https://github.com/Anshu666666/shieldApi>

Key Source Files Reference

File	Description
apps/gateway/app/main.py	FastAPI application entry point
apps/gateway/app/router.py	Proxy router with plugin pipeline
apps/gateway/app/admin.py	Admin REST API
apps/gateway/app/plugins/ip_blacklist.py	IP Blacklist Validator Plugin
apps/gateway/app/plugins/api_key.py	API Key Validator Plugin
apps/gateway/app/plugins/rate_limiter.py	Token Bucket Rate Limiter Plugin
apps/gateway/app/logging_emitter.py	Async non-blocking log writer
apps/anomaly-engine/engine/main.py	4-stage Anomaly Guardian Pipeline
packages/shared/redis_manager.py	Centralized Redis management
packages/shared/token_bucket.lua	Atomic Redis Lua rate limiting script
apps/dashboard/src/	React Admin Command Center
docker-compose.yml	Multi-container orchestration

File	Description
test_e2e_integration.py	Automated end-to-end integration tests
test_redis_manager.py	Redis unit tests